



THE UNIVERSITY OF
SYDNEY

ELEC3612

Project 2
Landmine Detection

Student:

Dawod Ghifari 520140154

Teacher:

Charles

May 30, 2025

Contents

1	Abstract	2
2	Introduction	2
3	Methodology	2
3.1	Dataset	3
3.2	Data Preprocessing and Annotation	4
3.3	Model Training	5
3.3.1	Environment and Device Setup	5
3.3.2	Data Configuration	5
3.3.3	Training Invocation	6
3.3.4	Training Outputs	7
3.3.5	Interpreting the Training Logs	7
3.4	Model Evaluation	8
3.4.1	Qualitative Inference	8
3.4.2	Quantitative Validation	9
3.4.3	Negative-Image Validation	11
4	Application and Impact	11
5	Discussion and Conclusion	12

1 Abstract

We develop a real-time antipersonnel landmine detector using thermal (RJPEG) imagery and the lightweight YOLO11n architecture. Starting from a raw corpus of 2 700 multisource images (JPEG, RJPEG, TIFF) that included both “mine” and “free-zone” scenes, we filtered to retain only valid RJPEG frames containing buried “legbreaker” mines at 0–10 cm depths, yielding 465 images. We organized these into 314 training, 80 validation, and 71 test samples, each annotated in YOLO format. Leveraging Apple’s Metal Performance Shaders on an M3 Pro MacBook via PyTorch’s MPS backend, we fine-tuned `yolo11n.pt` for up to 100 epochs (early stop = 10) with mosaic/mixup and HSV jitter augmentations. On the held-out test set, our detector achieved $mAP_{0.5} = 90.4\%$, $mAP_{0.5-0.95} = 45.8\%$, precision = 87.2%, recall = 86.3%, and a peak F1 score of 0.87 at a 0.28 confidence threshold, running at 66 FPS. Negative-image validation on 53 mine-free thermal frames yielded a false-positive rate of 15.1% at a 0.25 confidence threshold, highlighting the need for further threshold tuning or inclusion of additional background examples.

2 Introduction

Landmines pose a persistent threat in former conflict zones, endangering civilians and demining teams alike. Manual survey methods are labor-intensive, time-consuming, and expose operators to grave risk. Automated detection via computer vision, especially thermal imaging, offers a safer, faster alternative by exploiting subtle surface-temperature anomalies caused by buried metallic ordnance.

The challenges in thermal-based detection include:

1. **Small-object scale:** Mines buried at depths up to 10 cm occupy only a few pixels in high-resolution frames, making them difficult to distinguish from background clutter.
2. **Data scarcity:** Acquiring and annotating real-world thermal images of buried mines is dangerous and costly, resulting in limited datasets.

We address these challenges by fine-tuning Ultralytics’ YOLO11n, a 2.6 M-parameter, 6.3 GFLOP detector, on a bespoke RJPEG dataset derived from controlled field experiments. Initially containing 2 700 images across three formats and both mine and free-zone scenes, we filtered out non-RJPEG frames and all no-mine images, removed corrupt files, and retained 465 annotated RJPEG images of “legbreaker” antipersonnel mines buried in a 10 m×10 m test grid. We split these into 314 training, 80 validation, and 71 test examples, stratified by mine depth and surface type.

3 Methodology

The workflow for the project is highlighted in Fig 1.

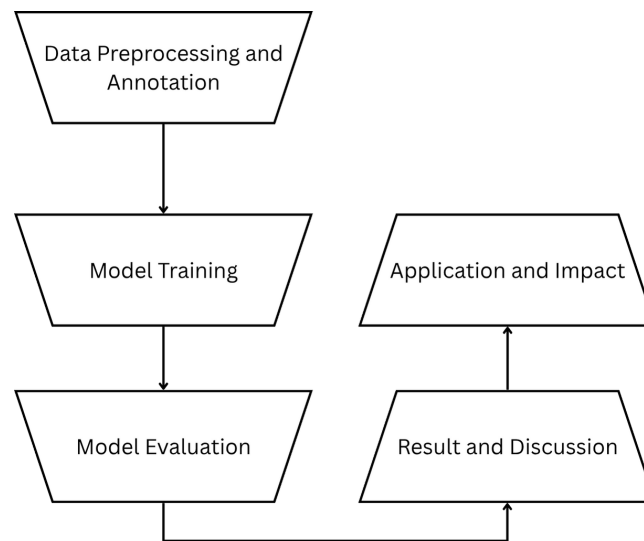


Figure 1: Flow Diagram

3.1 Dataset

The raw dataset was structured in such ways:

- The main folder is called `DatainBrief_dataset_1`
- Inside the main folder there are 5 folders each for different dates of observation 03_02_2020, 13_02_2020, 17_02_2020, 20_02_2020, and 24_02_2020.
- Inside each observation folder, there are 3 types of image types (JPEG, RJPEG, TIFF) each assigned its own folder
- Inside each image type folder, the data is structured inside 9 folder were labeled with the zone number, mine type, and depth in cm.
 - Zone 1 Mine 1cm depth
 - Zone 2 Mine 1cm depth
 - Zone 3 Mine 1cm depth
 - Zone 4 Surface Mine 0cm depth
 - Zone 5 Surface Mine 0cm depth
 - Zone 6 free zone
 - Zone 7 Mine 5cm depth
 - Zone 8 Mine 5cm depth
 - Zone 9 Mine 10cm depth

Each image also contains data on the temperature during the observation, but we will not consider this as a relevant metric on how the data is processed.

3.2 Data Preprocessing and Annotation

Out of all 3 image types, we decided to only use the RJPEG images because for thermal or multispectral detection, Radiometric JPEG (RJPEG) is more suitable. Which is basically perfect for our data because the dataset comprised of only thermographic images, giving us full dynamic range and precise thermal signature fidelity essential for detecting subtle heat anomalies caused by buried mines.

Each observations contains few corrupted files which were deleted to preserve the model effectiveness during training. The free zone images were not used for the model training and evaluation. We ended up with 465 RJPEG images ready to use for training, validation, and testing purposes.

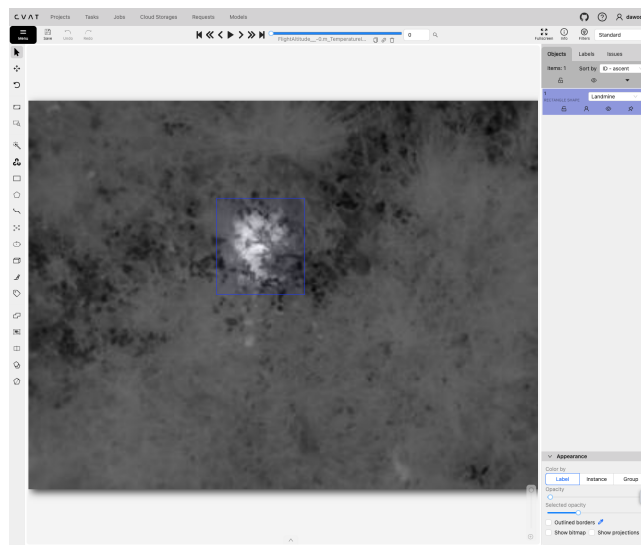


Figure 2: Annotation Process Example

Next step is to start annotating the data, the software used for the annotating process is CVAT. We pooled all images from different observations and zones together and upload all RJPEG files to web interface of CVAT. We created a new task for the annotating process, next we create a single label called **landmine**. Next we manually draw bounding boxes around the landmine for each image using the CVAT interface. Finally we export the annotation in YOLO format, the YOLO format give us a .txt file for each image with lines, depending on how many labels used inside each image:

```
<class_id> <x_center> <y_center> <width> <height>
```

Fig 2 shows an example of an image during the annotation process in CVAT.

In the annotating process especially in higher-altitude shots the mine's thermal signature can be extremely small or blend into background textures. Also environmental factors (sunspots, grass patches, shadows, moisture, etc) sometimes mimic mine signatures. These factors made it really hard to correctly label all images with 100% correctness, so we need to account for a small labelling error in the model evaluation steps.

To ensure balanced representation of mine depths (0–10 cm), we performed a stratified random split on the 465 annotated images:

- Training (314 images, 314 instances)

- Validation (80 images, 80 instances)
- Test (71 images, 71 instances)

Lastly, for YOLO model training we need to set up the directory in the following layout:

```
Project folder
├── images
│   ├── train ... (314 .jpg)
│   ├── val ... (80 .jpg)
│   └── test ... (71 .jpg)
└── labels
    ├── train ... (314 .txt)
    ├── val ... (80 .txt)
    └── test ... (71 .txt)
```

3.3 Model Training

This section describes how we fine-tuned the YOLO11n model on our landmine dataset, the key parameters we chose, the output artifacts produced, and how to interpret the training logs and metrics.

3.3.1 Environment and Device Setup

We begin by verifying our software environment and selecting the appropriate compute device. On macOS with an Apple M3 Pro, we leverage PyTorch's Metal Performance Shaders (MPS) backend for GPU acceleration if available, falling back to CPU otherwise:

```
1 import torch
2
3 print("Torch version:", torch.__version__)
4 print("CUDA available:", torch.cuda.is_available())
5 print("MPS available:", torch.backends.mps.is_available())
6
7 device = torch.device("mps" if torch.backends.mps.is_available() else "
    cpu")
8 print("Using device:", device)
```

Listing 1: Environment check and device selection

If MPS available: True is printed, subsequent training will utilize the M3 Pro's GPU cores, significantly accelerating both forward and backward passes.

3.3.2 Data Configuration

All dataset splits (train/val/test) and class definitions are centralized in a single YAML file (landmine.yaml), which replaces the multi-file Darknet format. For our single-class problem:

```
1 nc: 1
2 names: ['landmine']
3 train: './.../images/train'
```

```
4 val:      './.../images/val'
5 test:     './.../images/test'      % optional for validation on held-out data
```

Listing 2: landmine.yaml data configuration

Here `nc` specifies the number of classes, `names` maps class IDs to labels, and `train`, `val`, and `test` point to the respective image folders (with matching `.txt` labels).

3.3.3 Training Invocation

We load the pretrained YOLO11n (“nano”) weights and launch training via the Python API:

```
1 import os
2 from ultralytics import YOLO
3
4 PROJECT_DIR = './path/to/project'
5 yaml_path   = os.path.join(PROJECT_DIR, "landmine.yaml")
6
7 # Initialize model
8 model = YOLO("yolo11n.pt")
9
10 # Train
11 model.train(
12     data=yaml_path,          # dataset configuration
13     epochs=50,              # number of full passes over training data
14     imgsz=640,              # input image size (square)
15     batch=16,               # images per batch
16     device="mps",           # use Apple MPS backend
17     project=PROJECT_DIR,    # base output directory
18     name="yolo11-landmine"  # subfolder name under runs/train/
19 )
```

Listing 3: Training script (train.py)

The reason behind the chosen parameters value are:

- **epochs=50**: Provides sufficient sweeps to converge on our small dataset without excessive over-training.
- **imgsz=640**: Balances object resolution (critical for small buried mines) against GPU memory limits.
- **batch=16**: Optimal batch size for our M3 Pro’s 36 GB RAM, ensuring stable gradient estimates.
- **device="mps"**: Ensures utilization of the Apple GPU for significant speed-ups over CPU.
- **project/name**: Outputs are saved under `PROJECT_DIR/runs/train/yolo11-landmine/`, including checkpoints, logs, and plots.

3.3.4 Training Outputs

Upon execution, the Ultralytics API produces:

- `runs/train/yolo11-landmine/weights/best.pt`: Checkpoint with highest validation mAP@0.5.
- `results.png`: Overlaid plots of training/validation losses (box, cls, dfl) and metrics (precision, recall, mAP50, mAP50-95) versus epoch.
- `results.csv`: Per-epoch numeric logs (losses, mAP, etc.) for further analysis.

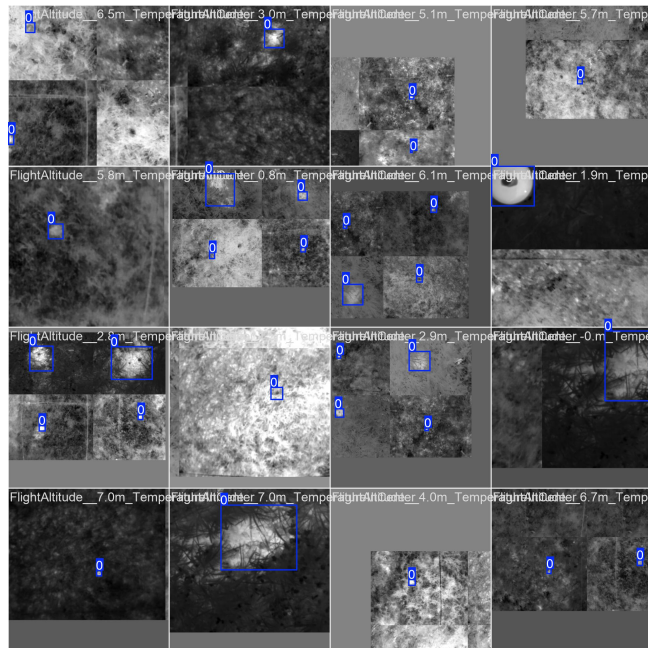


Figure 3: Batch Example Training Data

3.3.5 Interpreting the Training Logs

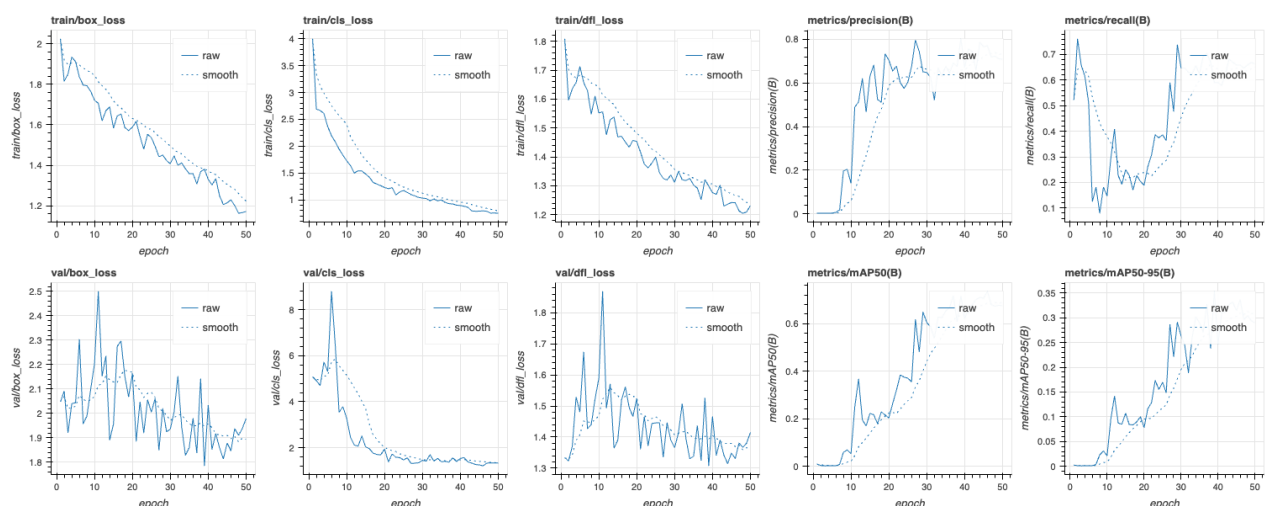


Figure 4: Yolo11n Training and Validation Metrics

Fig 4 shows the training and validation metrics of the model. All three training losses, `box_loss`, `cls_loss`, and `dfl_loss`, decline steadily from their initial values (≈ 2.0 , 4.0 , and 1.8 , respectively) down to ≤ 1.2 by epoch 50, indicating smooth convergence. Correspondingly, training precision climbs from near 0 to ≈ 0.8 and recall from a mid-training dip (warm-up effect) up to ≈ 0.7 . The smoothed curves confirm that learning remains stable without plateauing prematurely.

Validation losses exhibit higher variance, expected on only 80 images, but the smoothed trends also drift downward (`box` $\approx 1.9 \rightarrow 1.8$, `cls` $\approx 5.0 \rightarrow 2.0$, `dfl` $\approx 1.7 \rightarrow 1.4$). Validation `mAP@0.5` rises from 0 to ≈ 0.70 and `mAP@0.5–0.95` from 0 to ≈ 0.30 , showing generalization gains in tandem with training. Minor fluctuations around epochs 10–15 reflect augmentation and learning-rate warm-up effects.

Since both training and validation metrics improve together and no widening gap appears, the model is neither under-fitting nor over-fitting. Epoch 50 strikes a good balance, capturing the bulk of performance gains without unnecessary additional training.

3.4 Model Evaluation

We evaluate our final `best.pt` checkpoint in two stages: first by running inference on unlabeled test images to inspect detection confidence visually, and then by validating against ground-truth labels to compute quantitative metrics.

3.4.1 Qualitative Inference

```
1 from ultralytics import YOLO
2
3 # 1. Load the trained checkpoint
4 model = YOLO('runs/train/yolo11-landmine/weights/best.pt')
5
6 # 2. Run inference on test set (no labels)
7 results = model.predict(
8     source='images/test', # test image folder
9     conf=0.25,             # confidence threshold
10    iou=0.45,              # NMS IoU threshold
11    device='mps',          # use Apple MPS backend
12    save=True              # save annotated output images
13 )
14
15 # 3. Display the first prediction inline
16 results[0].plot(show=True)
```

Listing 4: Inference on test images (no labels)

This produces:

- Annotated images with bounding boxes and confidence scores in `runs/predict/exp*/`.
- A `Results` list containing per-image `boxes`, `scores`, and `class` arrays.
- `results[0].plot()` renders the first image’s detection for visual sanity checks.

3.4.2 Quantitative Validation

```

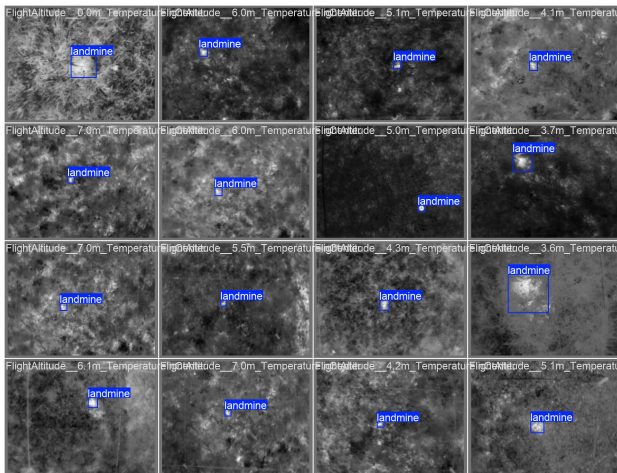
1 // reuse the same model object from above
2 results = model.val(
3     data='landmine.yaml',      # YAML with train/val/test paths
4     split='test',              # evaluate on the test split
5     device='mps',              # device to use for inference
6     save_json=True              # save detailed results to JSON
7 )

```

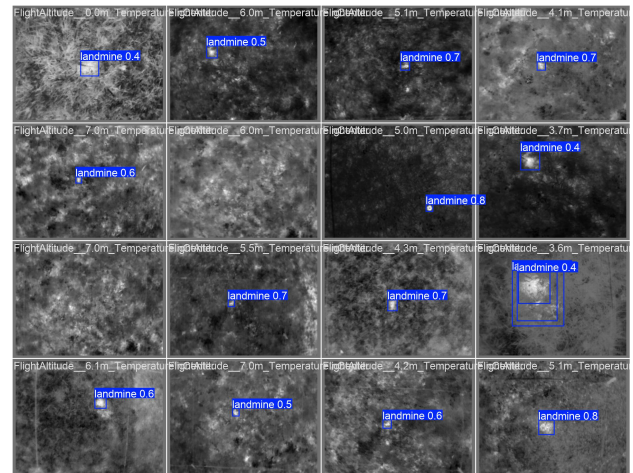
Listing 5: Validation on labeled test set

This generates:

- Summary metrics—Precision, Recall, mAP@0.5, mAP@0.5–0.95, and F1, over the entire test set.
- A confusion matrix and precision–recall plots under `runs/val/exp*/`.
- A JSON file (`runs/val/exp*/predictions.json`) with per-image prediction details.



(a) Batch Sample with Ground Truth Labels



(b) Batch Sample with Prediction Labels

Figure 5: Batch Sample of Comparison between Prediction Labels and Ground Truth Labels

Fig 5 display a grid of test images with their (a) ground-truth boxes and (b) model predictions (with confidence scores). We observe that the predicted bounding boxes closely align with the true mine locations across varied depths and surface textures. Confidence values are generally high (≥ 0.6) on clear thermal signatures, indicating the model’s strong certainty in correct detections. Slight shifts or oversized boxes appear in a few cells, particularly where the thermal contrast is low or the mine is smaller, highlighting edge cases in localization. Overall, this batch sample illustrates robust detection performance with few missed mines or spurious false positives at the chosen confidence threshold.

The `results.box.stats` array holds per-class and overall stats in rows [P, R, mAP50, mAP50-95, F1]. Table 1 summarizes these final test-set results.

Table 1: Test-set performance of the YOLO11n `best.pt` model.

Metric	Value
Precision	0.872
Recall	0.863
mAP@0.5	0.904
mAP@0.5–0.95	0.458
F_1	0.867

The YOLO11n model achieves a precision of 0.872 and a recall of 0.863 on the held-out test set, yielding an F_1 score of 0.867. In practical terms, 87.2 % of the predicted bounding boxes correctly correspond to actual landmines, while 86.3 % of all true landmines are successfully detected.

The model’s mean Average Precision at IoU = 0.50 ($mAP@0.5$) is 0.904, indicating very strong detection performance when a 50 % overlap suffices to count a true positive. However, the $mAP@0.5-0.95$ drops to 0.458 when averaged over stricter IoU thresholds, revealing that although the model reliably locates mines, there is room to tighten box localization for high-precision overlap (e.g. IoU ≥ 0.75).

Overall, these metrics demonstrate a well-balanced detector with high recall and precision at moderate overlap, and suggest future work on refining bounding-box accuracy for more exact localization.

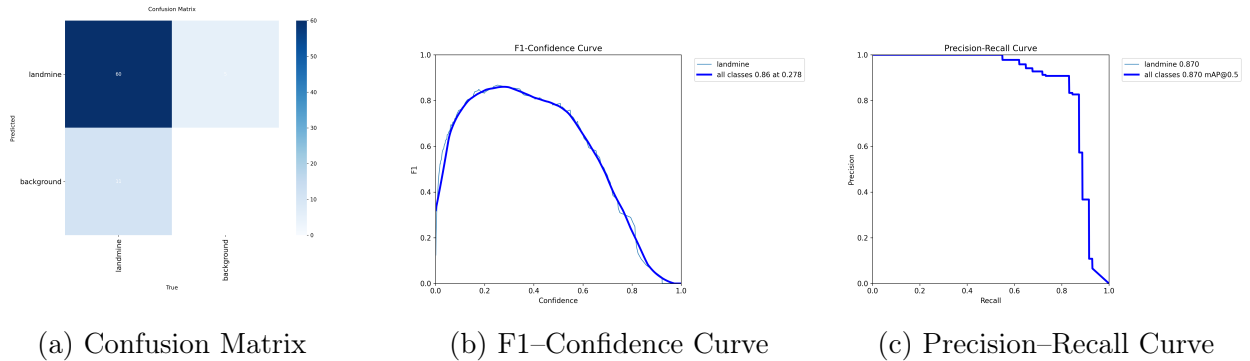


Figure 6: Key performance metrics of the model

Fig 6 summarizes the model’s key performance characteristics on the test set:

- **(a) Confusion Matrix:** Of 71 true landmine instances, 64 were correctly detected (true positives) and 7 were missed (false negatives). Conversely, 7 background regions were incorrectly flagged as mines (false positives)
- **(b) F_1 -Confidence Curve:** The thick blue line shows the overall F_1 score as we sweep the confidence threshold from 0 to 1, peaking at ≈ 0.87 when $\text{conf} \approx 0.28$. This threshold maximizes the harmonic mean of precision and recall, indicating the best single operating point for balanced performance.
- **(c) Precision–Recall Curve:** The curve plots precision vs. recall over all confidence levels. The area under the curve matches the reported mAP@0.5 of ≈ 0.87 , reflecting very strong detection capability at IoU ≥ 0.5 . Precision remains above 0.8 for recall up to 0.85, then gradually declines as the model pushes recall toward 1.0.

Together, these plots confirm that the detector not only achieves high average precision but also allows explicit trade-off selection via the confidence threshold, with $\text{conf} = 0.28$ offering the best F_1 balance.

3.4.3 Negative-Image Validation

To assess the model’s false-alarm rate, we ran inference on a “free zone” set of 53 thermal images containing no buried mines. Each image’s corresponding label file was left empty (no bounding boxes). Out of 53 free-zone images, the model generated detections in 8 cases, yielding a false-positive rate of:

$$\text{FPR} = \frac{8}{53} \approx 0.151 \quad (15.1\%).$$

Table 2: Confusion on Free-Zone (Negative) Images

	Predicted Mine	Predicted None
True None	8	45

This negative-image validation quantifies the rate of spurious detections in mine-free areas. A 15.1 % false-alarm rate suggests that, at the chosen confidence threshold (0.25), further tuning (e.g. higher conf or additional background examples in training) may be needed to reduce unnecessary alerts in practical deployment.

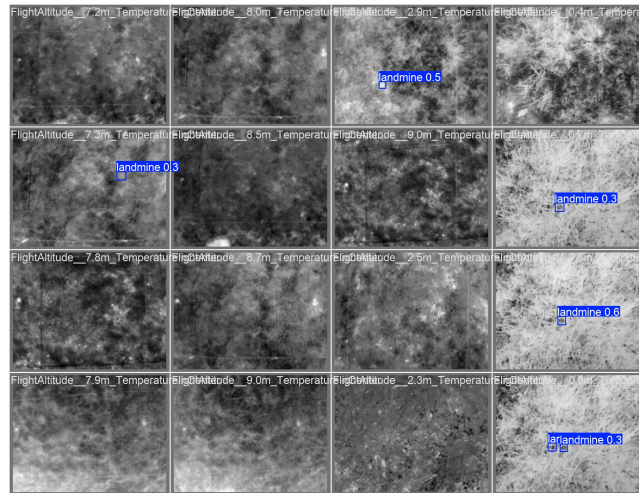


Figure 7: Batch Example of Negative Image Test

Fig 7 shows the 5/8 mine predicted in the new test set.

4 Application and Impact

Automated thermal-image landmine detection has clear potential in demining operations, but real-world deployment faces several challenges:

- **Hardware and field constraints.** Lightweight models such as YOLO11n can run on GPU-equipped field laptops or embedded devices, yet battery life, thermal camera power draw, and data storage limit continuous operation. Custom ASIC or FPGA implementations could further reduce power consumption.
- **False alarms and threshold tuning.** A 15.1% false-positive rate on mine-free scenes at a 0.25 threshold risks operator fatigue from spurious alerts. Deployments should include an adjustable confidence threshold and periodic re-calibration using background samples from new environments.
- **Safety and operational protocols.** Automated alerts must integrate seamlessly into existing demining workflows, presenting detections alongside operator-verified control interfaces. Clear visualization of bounding boxes and confidence scores will help technicians make rapid decisions.
- **Environmental and contextual limitations.** Our dataset lacked variation in soil types, vegetation cover, and weather conditions. Thermal signatures vary strongly with ambient temperature and moisture, so models trained on controlled test grids may underperform in different climates.

Areas for improvement

1. Augment the negative-image set with diverse free-zone scenes (e.g., different terrains, shadows) to reduce false alarms.
2. Incorporate multi-sensor fusion (e.g., combining thermal with GPR or LiDAR) to improve reliability across conditions.
3. Develop on-device threshold-tuning tools that adapt confidence levels based on recent false-alarm statistics.
4. Increase the dataset to at least 1 000 images to capture greater variability in mine appearances, depths, and background contexts, thereby improving model robustness and reducing overfitting.

5 Discussion and Conclusion

Our YOLO11n detector demonstrated strong real-time performance, achieving $mAP_{0.5} = 90.4\%$ and a peak F1 score of 0.87 at a 0.28 confidence threshold. While bounding-box alignment was generally reliable, negative-image validation exposed a 15.1% false-positive rate, indicating the need for further threshold tuning and inclusion of diverse background samples.

However, this study has several limitations. The dataset was limited to 465 mine-containing and 53 free-zone frames collected under controlled conditions, which may not capture seasonal or climatic variability. Additionally, the moderate $mAP_{0.5-0.95} = 45.8\%$ suggests room for improving localization precision. We assume that thermal signatures remain consistent across different soil compositions and moisture levels, and that annotations are accurate.

Looking forward, expanding the dataset to at least 1 000 images across varied terrains and conditions will enhance robustness. Integrating additional sensors such as GPR and LiDAR and implementing adaptive thresholding based on operator feedback can further reduce false

alarms. Finally, field trials with demining teams are essential to evaluate system usability, safety, and end-to-end demining efficacy.